

Part 1: Data Preprocessing & Exploratory Data Analysis (EDA)

=====

STUDENT INFORMATION (REQUIRED - DO NOT DELETE)

=====

Student 1

BITS ID: 2025AB05033

Name: VAIBHAV BHANDEO

Email: 2025AB05033@wilp.bits-pilani.ac.in

Student 2

BITS ID: 2025aa05448

Name: VAIBHAVI VISHWANATH
BADIGER

Email: 2025aa05448@wilp.bits-pilani.ac.in

Group No: 93

Subject: NLP

Date: 01-June-2026

Student 3

BITS ID: 2025aa05606

Name: V Raj Kumar

Email: 2025aa05606@wilp.bits-pilani.ac.in

Group No: 93

Subject: NLP

Date: 01-June-2026

Student 4

BITS ID: 2025AA05610

Name: VSSGG Rahul Mugada

Email: 2025AA05610@wilp.bits-pilani.ac.in

Group No: 93

Subject: NLP

Date: 01-June-2026

Student 5

BITS ID: 2025aa05165

Name: Rohit Vadge

Email: 2025aa05165@wilp.bits-pilani.ac.in

Group No: 93

Subject: NLP

Date: 01-June-2026

```
In [1]: # -----  
  
# CELL 0 – Virtual Lab Header  
  
# Prints execution timestamp and VM identifier as required by the  
# assignment submission guidelines.  
  
# -----  
  
import datetime  
import getpass  
import platform  
import socket
```

```

print("=" * 60)
print("BITS VIRTUAL LAB EXECUTION HEADER")
print("=" * 60)
print(f"Timestamp      : {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f"User              : {getpass.getuser()}")
print(f"Hostname (VM)     : {socket.gethostname()}")
print(f"Platform          : {platform.system()} {platform.release()}")
print(f"Python Version:   : {platform.python_version()}")
print("=" * 60)

```

```

=====
BITS VIRTUAL LAB EXECUTION HEADER
=====

```

```

Timestamp      : 2026-06-13 15:35:24
User           : cloud
Hostname (VM)  : 2025ab05033
Platform       : Linux 5.14.0-503.14.1.el9_5.x86_64
Python Version: 3.12.7
=====

```

```

In [2]: import sys
import sklearn
import sklearn.metrics

print("Success! Python version:", sys.version)
print("Scikit-Learn location:", sklearn.__file__)

```

```

Success! Python version: 3.12.7 | packaged by Anaconda, Inc. | (main, Oct 4
2024, 13:27:36) [GCC 11.2.0]
Scikit-Learn location: /home/cloud/anaconda3/lib/python3.12/site-packages/sk
learn/__init__.py

```

```

In [3]: # Install NLTK if not already installed
%pip install scikit-learn numpy

import nltk
from nltk.corpus import brown
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from collections import defaultdict

# Download the Brown corpus if not already downloaded
nltk.download('brown')
nltk.download('universal_tagset')

```

```
Requirement already satisfied: scikit-learn in /home/cloud/anaconda3/lib/python3.12/site-packages (1.9.0)
Requirement already satisfied: numpy in /home/cloud/anaconda3/lib/python3.12/site-packages (1.26.4)
Requirement already satisfied: scipy>=1.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.4.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: narwhals>=2.0.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (2.22.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
[nltk_data] Downloading package brown to /home/cloud/nltk_data...
[nltk_data] Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to
[nltk_data] /home/cloud/nltk_data...
[nltk_data] Package universal_tagset is already up-to-date!
```

Out[3]: True

a. Data Loading

Loading the specified categories from the NLTK Brown corpus. The assignment specifies Training Category: news and Testing Category: fiction.

```
In [4]: # Load the Brown corpus with universal tagset
# Training data: 'news' category
train_sentences = brown.tagged_sents(categories='news', tagset='universal')

# Testing data: 'fiction' category
test_sentences_raw = brown.tagged_sents(categories='fiction', tagset='universal')

print(f"Number of training sentences: {len(train_sentences)}")
print(f"Number of testing sentences (raw): {len(test_sentences_raw)}")

# Display a sample training sentence
print("\nSample training sentence:")
print(train_sentences[0])

# Display a sample testing sentence
print("\nSample testing sentence:")
print(test_sentences_raw[0])
```

Number of training sentences: 4623
 Number of testing sentences (raw): 4249

Sample training sentence:

```
[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'),
 ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'), ('investigation', 'NOUN'), ('of', 'ADP'), ('Atlanta's', 'NOUN'), ('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('`', 'PUNCT'), ('.', 'PUNCT'), ('no', 'DET'), ('evidence', 'NOUN'), ('"', 'PUNCT'), ('.', 'PUNCT'), ('that', 'ADP'), ('any', 'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.', 'PUNCT'), ('.', 'PUNCT')]
```

Sample testing sentence:

```
[('Thirty-three', 'NUM')]
```

b. Exploratory Data Analysis (EDA)

This section performs EDA on the loaded Brown corpus data. We will analyze POS tag distribution and sentence length distribution.

```
In [5]: # Function to extract all words and tags from a tagged corpus
def get_words_and_tags(tagged_sentences):
    words = []
    tags = []
    for sent in tagged_sentences:
        for word, tag in sent:
            words.append(word.lower())
            tags.append(tag)
    return words, tags

# Get words and tags for training and testing data
train_words, train_tags = get_words_and_tags(train_sentences)
test_words, test_tags = get_words_and_tags(test_sentences_raw)

print(f"Total words in training set: {len(train_words)}")
print(f"Total unique words in training set: {len(set(train_words))}")
print(f"Total POS tags in training set: {len(set(train_tags))}")

print(f"\nTotal words in testing set: {len(test_words)}")
print(f"Total unique words in testing set: {len(set(test_words))}")
print(f"Total POS tags in testing set: {len(set(test_tags))}")
```

Total words in training set: 100554
 Total unique words in training set: 13112
 Total POS tags in training set: 12

Total words in testing set: 68488
 Total unique words in testing set: 8680
 Total POS tags in testing set: 12

```
In [6]: # 1. POS Tag Distribution (Training Data)
plt.figure(figsize=(15, 7))
pos_counts = pd.Series(train_tags).value_counts()
sns.barplot(x=pos_counts.index, y=pos_counts.values)
plt.title('Distribution of POS Tags in Training Data (News Category)')
```

```

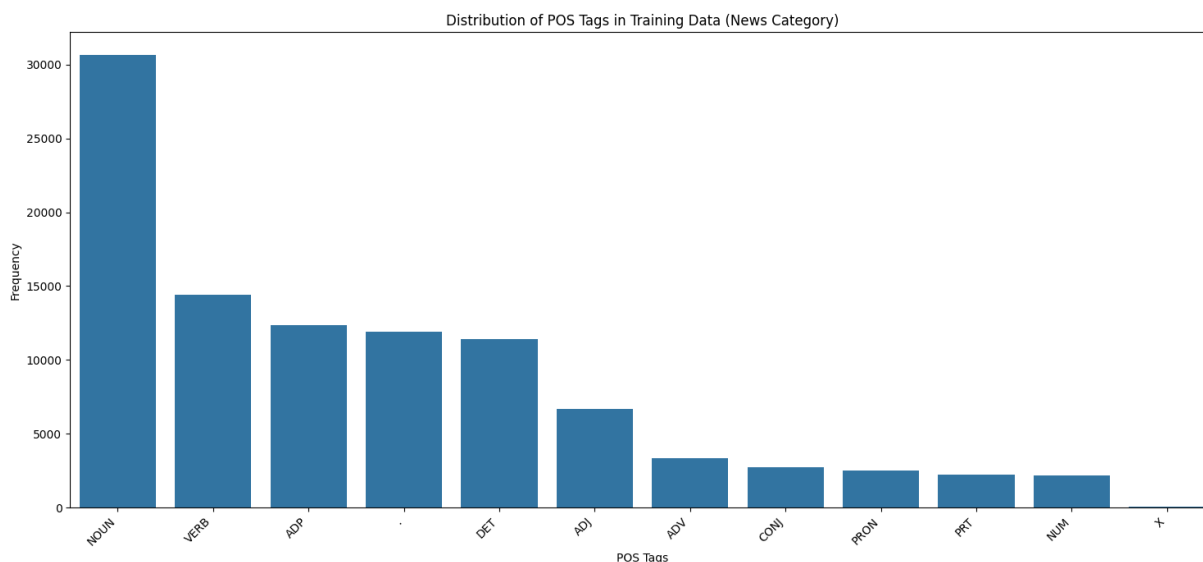
plt.xlabel('POS Tags')
plt.ylabel('Frequency')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

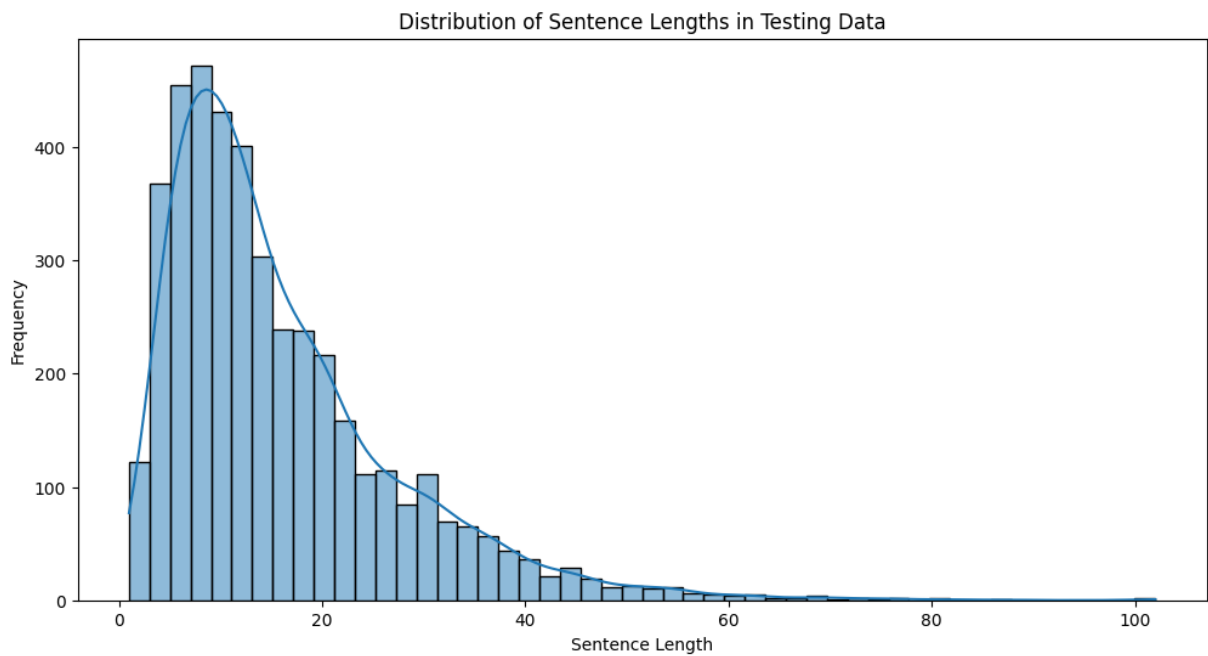
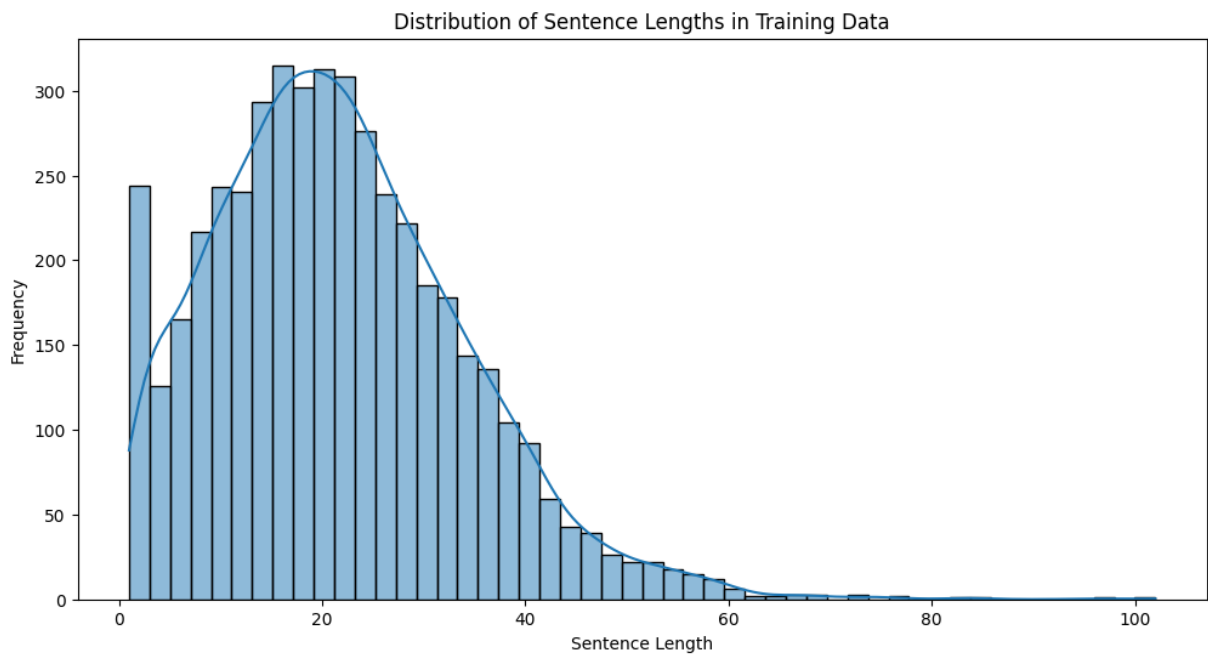
# 2. Sentence Length Distribution (Training Data)
train_sentence_lengths = [len(s) for s in train_sentences]
plt.figure(figsize=(12, 6))
sns.histplot(train_sentence_lengths, bins=50, kde=True)
plt.title('Distribution of Sentence Lengths in Training Data')
plt.xlabel('Sentence Length')
plt.ylabel('Frequency')
plt.show()

# 3. POS Tag Distribution (Testing Data)
plt.figure(figsize=(15, 7))
pos_counts_test = pd.Series(test_tags).value_counts()
sns.barplot(x=pos_counts_test.index, y=pos_counts_test.values)
plt.title('Distribution of POS Tags in Testing Data (Fiction Category)')
plt.xlabel('POS Tags')
plt.ylabel('Frequency')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# 4. Sentence Length Distribution (Testing Data)
test_sentence_lengths = [len(s) for s in test_sentences_raw]
plt.figure(figsize=(12, 6))
sns.histplot(test_sentence_lengths, bins=50, kde=True)
plt.title('Distribution of Sentence Lengths in Testing Data')
plt.xlabel('Sentence Length')
plt.ylabel('Frequency')
plt.show()

```





c. Vocabulary Handling & Unknown Words

In this section, we build the vocabulary from the training set and identify all unique POS tags. This is crucial for constructing the HMM matrices. We'll also prepare a strategy for handling unknown words that appear in the test set but not in the training set.

```
In [7]: # Create vocabulary from training data
# Use a set for efficient lookup
vocabulary = set(train_words)

# Get all unique POS tags from the training data
# This will represent the hidden states in our HMM
all_pos_tags = list(set(train_tags))
all_pos_tags.sort() # Sort for consistent ordering

print(f"Size of vocabulary (unique words from training data): {len(vocabulary)}")
print(f"Number of unique POS tags: {len(all_pos_tags)}")
print("\nSample of unique POS tags:")
print(all_pos_tags[:10]) # Print first 10 tags for brevity
```

Size of vocabulary (unique words from training data): 13112

Number of unique POS tags: 12

Sample of unique POS tags:

['.', 'ADJ', 'ADP', 'ADV', 'CONJ', 'DET', 'NOUN', 'NUM', 'PRON', 'PRT']

To handle unknown words (Out-Of-Vocabulary or OOV words) in the test set, we will replace them with a special token, typically `"_UNKNOWN_"`. This allows the HMM to assign probabilities to these words during inference.

```
In [8]: # Define a special token for unknown words
UNKNOWN_TOKEN = '_UNKNOWN_'

# Function to replace OOV words in sentences
def handle_oov_words(sentences, vocab):
    processed_sentences = []
    for sent in sentences:
        processed_sent = []
        for word, tag in sent:
            # Convert word to lowercase for consistency
            processed_word = word.lower()
            if processed_word not in vocab:
                processed_sent.append((UNKNOWN_TOKEN, tag))
            else:
                processed_sent.append((processed_word, tag))
        processed_sentences.append(processed_sent)
    return processed_sentences

# Apply OOV handling to the test sentences
test_sentences_processed = handle_oov_words(test_sentences_raw, vocabulary)

# Add UNKNOWN_TOKEN to the vocabulary for proper emission probability calcul
```

```

vocabulary.add(UNKNOWN_TOKEN)

print(f"Test sentences after OOV handling (first sentence):")
print(test_sentences_processed[0])
print(f"\nNew vocabulary size after adding UNKNOWN_TOKEN: {len(vocabulary)}"

# Check for OOV words in the first few test sentences
oov_count = 0
for i, (word, tag) in enumerate(test_sentences_processed[0]):
    if word == UNKNOWN_TOKEN:
        oov_count += 1
        print(f"  OOV word found: {test_sentences_raw[0][i][0]} replaced by

if oov_count == 0:
    print("No OOV words found in the first test sentence.")

```

Test sentences after OOV handling (first sentence):
 [('_UNKNOWN_', 'NUM')]

New vocabulary size after adding UNKNOWN_TOKEN: 13113
 OOV word found: Thirty-three replaced by _UNKNOWN_

d. Sentence Padding

To handle the start and end of sentences in our HMM, we will pad each sentence with special tokens: ('<s>', '<s>') for the beginning and ('</s>', '</s>') for the end. This allows for proper calculation of initial and final state transition probabilities.

```

In [9]: # Define start and end tokens for sentence padding
START_TOKEN = '<s>'
END_TOKEN = '</s>'

# Function to pad sentences with start and end tokens
def pad_sentences(sentences):
    padded_sentences = []
    for sent in sentences:
        padded_sentences.append([(START_TOKEN, START_TOKEN)] + sent + [(END_TOKEN, END_TOKEN)])
    return padded_sentences

# Apply padding to training and processed test sentences
train_sentences_padded = pad_sentences(train_sentences)
test_sentences_processed_padded = pad_sentences(test_sentences_processed)

print(f"Original first training sentence: {train_sentences[0]}\n")
print(f"Padded first training sentence: {train_sentences_padded[0]}\n")

print(f"Original first test sentence: {test_sentences_raw[0]}\n")
print(f"Padded first test sentence (after OOV handling): {test_sentences_pro

```

```
Original first training sentence: [('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'), ('investigation', 'NOUN'), ('of', 'ADP'), ('Atlanta's', 'NOUN'), ('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('`', '.'), ('no', 'DET'), ('evidence', 'NOUN'), ('`', '.'), ('that', 'ADP'), ('any', 'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.', '.')]

```

```
Padded first training sentence: [('<s>', '<s>'), ('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'), ('investigation', 'NOUN'), ('of', 'ADP'), ('Atlanta's', 'NOUN'), ('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('`', '.'), ('no', 'DET'), ('evidence', 'NOUN'), ('`', '.'), ('that', 'ADP'), ('any', 'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.', '.'), ('</s>', '</s>')]

```

```
Original first test sentence: [('Thirty-three', 'NUM')]

```

```
Padded first test sentence (after OOV handling): [('<s>', '<s>'), ('_UNKNOWN_', 'NUM'), ('</s>', '</s>')]

```

e. Data Splitting

The assignment specifies using the 'news' category for training and 'fiction' for testing. We have already loaded these categories separately. For the purpose of the HMM, the training data will be `train_sentences_padded`, and the test data will be `test_sentences_processed_padded`.

```
In [10]: # The data has already been split and prepared:
# Training data: train_sentences_padded
# Testing data: test_sentences_processed_padded

print(f"Final number of training sentences (padded): {len(train_sentences_padded)}")
print(f"Final number of testing sentences (padded and OOV-handled): {len(test_sentences_processed_padded)}")

print("\nData preprocessing and splitting complete. Ready for Model Architecture & Parameter Estimation.")

```

```
Final number of training sentences (padded): 4623

```

```
Final number of testing sentences (padded and OOV-handled): 4249

```

Data preprocessing and splitting complete. Ready for Model Architecture & Parameter Estimation.

Part 2: Model Architecture & Parameter Estimation

This section focuses on building a fundamental Bigram Hidden Markov Model from scratch. We will estimate the Transition Probability Matrix (A) and Emission Probability Matrix (B) using Maximum Likelihood Estimation (MLE) from the training data.

Estimate the Transition Probability Matrix (A)

The Transition Probability Matrix A_{ij} represents the probability of transitioning from state (POS tag) i to state (POS tag) j . We estimate this using Maximum Likelihood Estimation (MLE) based on the observed tag sequences in the training data:

$$A_{ij} = P(tag_j | tag_i) = \frac{Count(tag_i, tag_j)}{Count(tag_i)}$$

We include the start (<s>) and end (</s>) tokens in our tag set to handle sentence boundaries.

```
In [11]: # Initialize counts for transitions
tag_tag_counts = defaultdict(lambda: defaultdict(int))
tag_counts = defaultdict(int)

# Populate tag_tag_counts and tag_counts from the training data
for sent in train_sentences_padded:
    for i in range(len(sent) - 1):
        current_tag = sent[i][1] # Current POS tag
        next_tag = sent[i+1][1] # Next POS tag

        tag_tag_counts[current_tag][next_tag] += 1
        tag_counts[current_tag] += 1

# Calculate Transition Probability Matrix (A) using log-probabilities
# A[current_tag][next_tag] = log(P(next_tag | current_tag))
# Initialize with -np.inf for unseen transitions
transition_probabilities = defaultdict(lambda: defaultdict(lambda: -np.inf))

for current_tag, next_tags in tag_tag_counts.items():
    total_current_tag_count = tag_counts[current_tag]
    if total_current_tag_count == 0: # Avoid division by zero
        continue
    for next_tag, count in next_tags.items():
        transition_probabilities[current_tag][next_tag] = np.log(count / tot
# Add a log(ALPHA) smoothing for potential transitions to UNKNOWN_TOKEN
# (though for tags, UNKNOWN_TOKEN is typically not a tag, so this is les

print("Sample Transition Log-Probabilities:")
```

```

print(f"log(P(NOUN | DET)): {transition_probabilities['DET']['NOUN']:.4f}")
print(f"log(P(VERB | NOUN)): {transition_probabilities['NOUN']['VERB']:.4f}")
print(f"log(P(ADP | VERB)): {transition_probabilities['VERB']['ADP']:.4f}")
print(f"log(P(NOUN | ADP)): {transition_probabilities['ADP']['NOUN']:.4f}")
print(f"log(P(END | .)): {transition_probabilities['.']['</s>']:.4f}")
print(f"log(P(DET | START)): {transition_probabilities['<s>']['DET']:.4f}")

# Convert to DataFrame for better visualization (optional but good for delivery)
# Convert back from log to probability for display purposes only
trans_df_display = pd.DataFrame({ct: {nt: np.exp(val) for nt, val in nts.items() if nt in ct} for ct in nts})
# Ensure all possible tags are in both index and columns for a complete matrix
all_tags_for_matrix = sorted(list(set(all_pos_tags + [START_TOKEN, END_TOKEN])))
trans_df_display = trans_df_display.reindex(index=all_tags_for_matrix, columns=all_tags_for_matrix)

print("\nTransition Probability Matrix (first 5x5 rows/cols - converted from log for display):")
print(trans_df_display.iloc[:5, :5])

plt.figure(figsize=(10, 8))
sns.heatmap(trans_df_display.iloc[:10, :10], annot=True, fmt=".2f", cmap="viridis")
plt.title('Heatmap of Transition Probabilities (Top 10x10 Tags)')
plt.xlabel('Next Tag')
plt.ylabel('Current Tag')
plt.show()

```

Sample Transition Log-Probabilities:

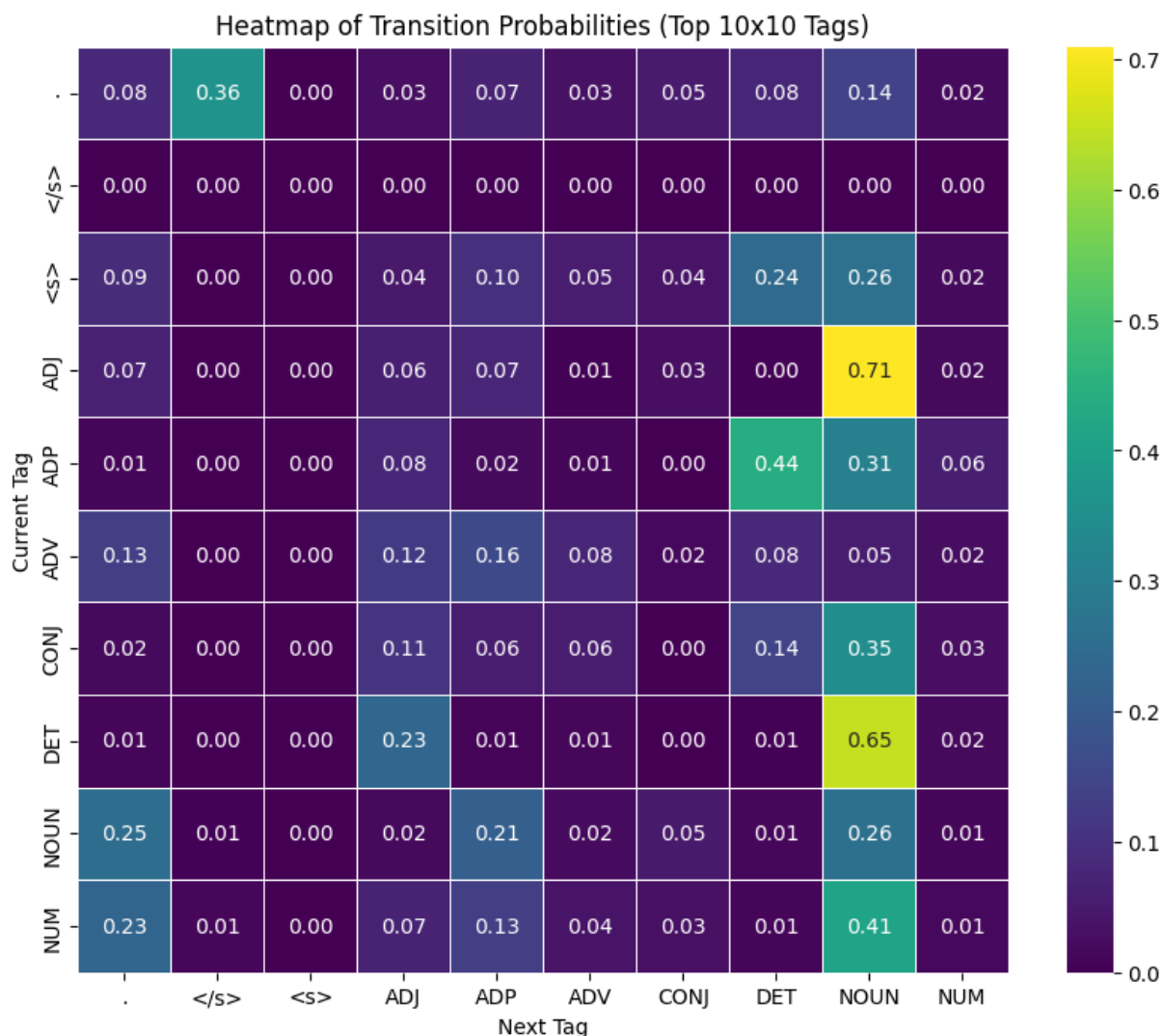
```

log(P(NOUN | DET)): -0.4350
log(P(VERB | NOUN)): -1.9908
log(P(ADP | VERB)): -1.7513
log(P(NOUN | ADP)): -1.1841
log(P(END | .)): -1.0101
log(P(DET | START)): -1.4213

```

Transition Probability Matrix (first 5x5 rows/cols - converted from log for display):

	.	</s>	<s>	ADJ	ADP
.	0.081489	0.364185	0.0	0.026660	0.066901
</s>	0.000000	0.000000	0.0	0.000000	0.000000
<s>	0.087605	0.000000	0.0	0.044992	0.098637
ADJ	0.065016	0.001939	0.0	0.060990	0.072771
ADP	0.008580	0.000081	0.0	0.077135	0.016835



Estimate the Emission Probability Matrix (B)

The Emission Probability Matrix B_{jk} represents the probability of observing a word k given a state (POS tag) j . Similar to the transition matrix, we estimate this using Maximum Likelihood Estimation (MLE) from the training data:

$$B_{jk} = P(\text{word}_k | \text{tag}_j) = \frac{\text{Count}(\text{tag}_j, \text{word}_k)}{\text{Count}(\text{tag}_j)}$$

We will also handle the `_UNKNOWN_` token here, assigning a small probability to it for all tags to avoid zero probabilities for unseen words in the test set.

```
In [12]: # Initialize counts for emissions
tag_word_counts = defaultdict(lambda: defaultdict(int))
tag_frequencies_for_emission = defaultdict(int) # New dictionary to correctl

# Populate tag_word_counts and tag_frequencies_for_emission
for sent in train_sentences_padded:
    for word, tag in sent:
        # Only process actual content words/tags for emission probabilities.
        # Special START_TOKEN and END_TOKEN do not emit words, so they are e
```

```

        if tag not in [START_TOKEN, END_TOKEN]:
            word_lower = word.lower()
            tag_word_counts[tag][word_lower] += 1
            tag_frequencies_for_emission[tag] += 1 # Count every occurrence

# Calculate Emission Probability Matrix (B) using log-probabilities
# B[tag][word] = log(P(word | tag))
# Initialize with -np.inf for unseen emissions
emission_probabilities = defaultdict(lambda: defaultdict(lambda: -np.inf))

ALPHA = 1e-6 # A small constant for smoothing unknown words
LOG_ALPHA = np.log(ALPHA) # Log of the smoothing constant

for tag, words_in_tag in tag_word_counts.items():
    total_tag_count = tag_frequencies_for_emission[tag] # Use the new, corre

    # This check ensures we don't divide by zero if, somehow, a tag with wor
    if total_tag_count == 0:
        # If a tag has no words, its emission probabilities should all be ve
        # This handles cases where a tag might only appear with START/END to
        for word in vocabulary: # Assign LOG_ALPHA to all words for this tag
            emission_probabilities[tag][word] = LOG_ALPHA
        emission_probabilities[tag][UNKNOWN_TOKEN] = LOG_ALPHA
        continue

    # Calculate probabilities for known words
    for word, count in words_in_tag.items():
        emission_probabilities[tag][word] = np.log(count / total_tag_count)

    # Handle UNKNOWN_TOKEN: assign a small, non-zero probability to it for e
    # This loop iterates over keys from tag_word_counts, which already exclu
    emission_probabilities[tag][UNKNOWN_TOKEN] = LOG_ALPHA # Assign log(ALPH

print("Sample Emission Log-Probabilities:")
print(f"log(P('the' | DET)): {emission_probabilities['DET']['the']:.4f}")
print(f"log(P('run' | VERB)): {emission_probabilities['VERB']['run']:.4f}")
print(f"log(P('car' | NOUN)): {emission_probabilities['NOUN']['car']:.4f}")
print(f"log(P('_UNKNOWN_' | NOUN)): {emission_probabilities['NOUN'][UNKNOWN_

# Convert to DataFrame for better visualization (optional)
# For emission matrix, it's very sparse due to large vocabulary, so showing

# Create a small sample DataFrame for common tags and words
sample_tags = ['DET', 'NOUN', 'VERB', 'ADJ', 'ADP']
sample_words = ['the', 'a', 'man', 'car', 'run', 'walk', 'good', 'bad', 'in']

emission_data = {word: [np.exp(emission_probabilities[tag][word]) if emissio
emission_df = pd.DataFrame(emission_data, index=sample_tags).fillna(0)

print("\nEmission Probability Matrix (Sample - converted from log for displa
print(emission_df)

```

Sample Emission Log-Probabilities:

```
log(P('the' | DET)): -0.5785
log(P('run' | VERB)): -6.8669
log(P('car' | NOUN)): -6.4185
log(P('_UNKNOWN_' | NOUN)): -13.815511
```

Emission Probability Matrix (Sample - converted from log for display):

	the	a	man	car	run	walk	good
\							
DET	0.560716	0.186145	0.000000	0.000000	0.000000	0.000000	0.000000
NOUN	0.000000	0.000261	0.002414	0.001631	0.000652	0.000196	0.000098
VERB	0.000000	0.000000	0.000000	0.000000	0.001042	0.000417	0.000000
ADJ	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.007009
ADP	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

	bad	in	on	_UNKNOWN_
\				
DET	0.000000	0.000000	0.000000	0.000001
NOUN	0.000000	0.000000	0.000000	0.000001
VERB	0.000000	0.000000	0.000000	0.000001
ADJ	0.000746	0.000000	0.000000	0.000001
ADP	0.000000	0.159773	0.053501	0.000001

Implement the Viterbi Algorithm

The Viterbi algorithm is a dynamic programming algorithm for finding the most likely sequence of hidden states (Viterbi path) that results in a sequence of observed events, especially in the context of Hidden Markov Models (HMMs).

Here's a high-level overview of the steps:

1. **Initialization:** For the first word, calculate the probability of each POS tag being the starting tag and emitting the first word.
2. **Recursion:** For each subsequent word, and for each possible current POS tag, calculate the maximum probability of reaching this tag, given all previous paths and the probability of emitting the current word.
3. **Termination:** Find the most probable sequence ending with the end-of-sentence token.
4. **Path Backtracking:** Reconstruct the sequence of POS tags by backtracking through the stored paths.

```
In [13]: def viterbi(words, all_pos_tags, transition_probabilities, emission_probabil
        """
        Implements the Viterbi algorithm to find the most likely sequence of POS
        This version uses log-probabilities to prevent numerical underflow.

        Args:
            words (list): A list of words (observations) in a sentence.
            all_pos_tags (list): A list of all possible POS tags (states), exclu
            transition_probabilities (defaultdict): log(P(tag_j | tag_i)).
            emission_probabilities (defaultdict): log(P(word_k | tag_j)).
```

vocabulary (set): Set of all known words, including UNKNOWN_TOKEN.
 START_TOKEN (str): Special token for the start of a sentence.
 END_TOKEN (str): Special token for the end of a sentence.
 UNKNOWN_TOKEN (str): Special token for out-of-vocabulary words.
 ALPHA (float): Small constant used for smoothing (will be converted

Returns:

tuple: A tuple containing the most likely sequence of POS tags and i
 ""

LOG_ALPHA = np.log(ALPHA)

Number of words in the sentence and number of possible POS tags

num_words = len(words)

*# Filter out START_TOKEN and END_TOKEN from all_pos_tags for Viterbi pro
 # as these are boundary tags and not actual POS tags for words.*

viterbi_tags = [tag for tag in all_pos_tags if tag not in [START_TOKEN,
 num_tags = len(viterbi_tags)

Create a mapping from tag string to its index for efficient NumPy arra

tag_to_idx = {tag: i for i, tag in enumerate(viterbi_tags)}

idx_to_tag = {i: tag for i, tag in enumerate(viterbi_tags)}

Initialize Viterbi table and backpointer table

viterbi_table[t][tag_idx] stores the maximum log-probability of a tag

backpointer[t][tag_idx] stores the index of the previous tag that led

viterbi_table = np.full((num_words, num_tags), -np.inf) *# Initialize wit*

backpointer = np.zeros((num_words, num_tags), dtype=int)

--- Initialization Step (t = 0) ---

For the first word, calculate the log-probability of each POS tag bein

*# log(P(tag_i | START_TOKEN) * P(word_0 | tag_i)) = log(P(tag_i | START_*

first_word = words[0].lower()

for i, current_tag in enumerate(viterbi_tags):

Log-probability of transitioning from START_TOKEN to current_tag

log_prob_start_to_tag = transition_probabilities[START_TOKEN].get(cu

Emission log-probability of the first word given the current_tag

If the word is OOV and becomes UNKNOWN_TOKEN, its log-probability

If the word is a known word not seen with this tag, use LOG_ALPHA

log_prob_word_given_tag = emission_probabilities[current_tag].get(fi

viterbi_table[0, i] = log_prob_start_to_tag + log_prob_word_given_ta

No backpointer for t=0

--- Recursion Step (t = 1 to num_words - 1) ---

For each subsequent word, calculate the maximum log-probability for ea

for t in range(1, num_words):

current_word = words[t].lower()

for j, current_tag in enumerate(viterbi_tags):

max_log_prob = -np.inf *# Initialize with negative infinity*

best_prev_tag_idx = 0

Emission log-probability of the current word given the current

```

log_prob_word_given_tag = emission_probabilities[current_tag].get(word, 0)

for i, prev_tag in enumerate(viterbi_tags):
    # Log-probability of path to previous tag + log-transition from prev_tag to current_tag
    log_prob = viterbi_table[t-1, i] + transition_probabilities[prev_tag, current_tag]

    if log_prob > max_log_prob:
        max_log_prob = log_prob
        best_prev_tag_idx = i

viterbi_table[t, j] = max_log_prob
backpointer[t, j] = best_prev_tag_idx

# --- Termination Step ---
# Find the maximum log-probability of a path ending with a transition to END_TOKEN
max_final_log_prob = -np.inf # Initialize with negative infinity
last_tag_of_best_path_idx = 0

for i, last_possible_tag in enumerate(viterbi_tags):
    # log(P(path_to_last_tag) * P(END_TOKEN | last_tag)) = log(P(path_to_last_tag) + log(P(END_TOKEN | last_tag)))
    log_prob = viterbi_table[num_words-1, i] + transition_probabilities[last_possible_tag, END_TOKEN]
    if log_prob > max_final_log_prob:
        max_final_log_prob = log_prob
        last_tag_of_best_path_idx = i

# --- Path Backtracking ---
# Reconstruct the best tag sequence by backtracking from the last tag
best_path_tags = [idx_to_tag[last_tag_of_best_path_idx]]
for t in range(num_words - 1, 0, -1):
    last_tag_of_best_path_idx = backpointer[t, last_tag_of_best_path_idx]
    best_path_tags.insert(0, idx_to_tag[last_tag_of_best_path_idx]) # Insert at beginning

return best_path_tags, np.exp(max_final_log_prob) # Return probability and best path tags

```

Part 3: Inference and Evaluation

a) Sequence Inference (POS Tagging)

Now, we will use our implemented Viterbi decoder to predict the POS tag sequences for selected sentences. We will pick two sentences from the training dataset and two from the test dataset.

```

In [14]: # Helper function to extract words from a tagged sentence for Viterbi input
def get_words_from_tagged_sentence(tagged_sentence_with_padding):
    # Filter out START_TOKEN and END_TOKEN and lowercase words
    return [word.lower() for word, tag in tagged_sentence_with_padding if tag != START_TOKEN and tag != END_TOKEN]

# Helper function to get actual tags from a tagged sentence for comparison
def get_actual_tags_from_tagged_sentence(tagged_sentence_with_padding):
    # Filter out START_TOKEN and END_TOKEN
    return [tag for word, tag in tagged_sentence_with_padding if tag != START_TOKEN and tag != END_TOKEN]

# Helper function to calculate token-level accuracy for a list of predicted

```

```

def calculate_accuracy(all_predicted_tags, all_actual_tags):
    correct_predictions = 0
    total_predictions = 0
    for pred_list, actual_list in zip(all_predicted_tags, all_actual_tags):
        for pred_tag, actual_tag in zip(pred_list, actual_list):
            if pred_tag == actual_tag:
                correct_predictions += 1
            total_predictions += 1
    return correct_predictions / total_predictions if total_predictions > 0

# --- Select 2 sentences from the training dataset --- #
# Sentence 1 from training data
train_sentence_1_raw = train_sentences_padded[10] # Example: index 10
train_words_1 = get_words_from_tagged_sentence(train_sentence_1_raw)
train_actual_tags_1 = get_actual_tags_from_tagged_sentence(train_sentence_1_raw)

# Sentence 2 from training data
train_sentence_2_raw = train_sentences_padded[100] # Example: index 100
train_words_2 = get_words_from_tagged_sentence(train_sentence_2_raw)
train_actual_tags_2 = get_actual_tags_from_tagged_sentence(train_sentence_2_raw)

# --- Select 2 sentences from the test dataset --- #
# Sentence 1 from test data (after OOV handling)
test_sentence_1_raw = test_sentences_processed_padded[0] # Example: index 0
test_words_1 = get_words_from_tagged_sentence(test_sentence_1_raw)
test_actual_tags_1 = get_actual_tags_from_tagged_sentence(test_sentence_1_raw)

# Sentence 2 from test data (after OOV handling)
test_sentence_2_raw = test_sentences_processed_padded[50] # Example: index 50
test_words_2 = get_words_from_tagged_sentence(test_sentence_2_raw)
test_actual_tags_2 = get_actual_tags_from_tagged_sentence(test_sentence_2_raw)

print("Selected Sentences and their Words for Viterbi:")
print(f"Train Sentence 1 (words): {train_words_1}")
print(f"Train Sentence 2 (words): {train_words_2}")
print(f"Test Sentence 1 (words): {test_words_1}")
print(f"Test Sentence 2 (words): {test_words_2}")

print("\n--- Running Viterbi on selected sentences ---")

# Run Viterbi for Training Sentence 1
predicted_tags_train_1, prob_train_1 = viterbi(train_words_1, all_pos_tags,
print(f"\nTraining Sentence 1: {train_words_1}")
print(f" Actual Tags: {train_actual_tags_1}")
print(f" Predicted Tags: {predicted_tags_train_1}")
print(f" Probability: {prob_train_1:.2e}")

# Run Viterbi for Training Sentence 2
predicted_tags_train_2, prob_train_2 = viterbi(train_words_2, all_pos_tags,
print(f"\nTraining Sentence 2: {train_words_2}")
print(f" Actual Tags: {train_actual_tags_2}")
print(f" Predicted Tags: {predicted_tags_train_2}")
print(f" Probability: {prob_train_2:.2e}")

# Run Viterbi for Test Sentence 1
predicted_tags_test_1, prob_test_1 = viterbi(test_words_1, all_pos_tags, tra

```

```
print(f"\nTest Sentence 1: {test_words_1}")
print(f"  Actual Tags:    {test_actual_tags_1}")
print(f"  Predicted Tags: {predicted_tags_test_1}")
print(f"  Probability: {prob_test_1:.2e}")

# Run Viterbi for Test Sentence 2
predicted_tags_test_2, prob_test_2 = viterbi(test_words_2, all_pos_tags, tra
print(f"\nTest Sentence 2: {test_words_2}")
print(f"  Actual Tags:    {test_actual_tags_2}")
print(f"  Predicted Tags: {predicted_tags_test_2}")
print(f"  Probability: {prob_test_2:.2e}")
```

Selected Sentences and their Words for Viterbi:

Train Sentence 1 (words): ['it', 'urged', 'that', 'the', 'city', '``', 'take', 'steps', 'to', 'remedy', 'this', 'problem', '.']

Train Sentence 2 (words): ['daniel', 'personally', 'led', 'the', 'fight', 'for', 'the', 'measure', 'which', 'he', 'had', 'watered', 'down', 'considerably', 'since', 'its', 'rejection', 'by', 'two', 'previous', 'legislatures', 'in', 'a', 'public', 'hearing', 'before', 'the', 'house', 'committee', 'on', 'revenue', 'and', 'taxation', '.']

Test Sentence 1 (words): ['_unknown_']

Test Sentence 2 (words): ['virginia', 'treated', 'him', 'with', 'attention', 'and', 'tried', 'to', '_unknown_', 'his', '_unknown_', 'with', 'special', 'food', ':', '_unknown_', 'cookies', '_unknown_', '--', 'the', 'result', 'of', 'devoted', 'hours', 'in', 'the', 'tiled', 'kitchen', '.']

--- Running Viterbi on selected sentences ---

Training Sentence 1: ['it', 'urged', 'that', 'the', 'city', '``', 'take', 'steps', 'to', 'remedy', 'this', 'problem', '.']

Actual Tags: ['PRON', 'VERB', 'ADP', 'DET', 'NOUN', '.', 'VERB', 'NOUN', 'PRT', 'VERB', '.', 'DET', 'NOUN', '.']

Predicted Tags: ['PRON', 'VERB', 'ADP', 'DET', 'NOUN', '.', 'VERB', 'NOUN', 'PRT', 'VERB', '.', 'DET', 'NOUN', '.']

Probability: 5.51e-37

Training Sentence 2: ['daniel', 'personally', 'led', 'the', 'fight', 'for', 'the', 'measure', 'which', 'he', 'had', 'watered', 'down', 'considerably', 'since', 'its', 'rejection', 'by', 'two', 'previous', 'legislatures', 'in', 'a', 'public', 'hearing', 'before', 'the', 'house', 'committee', 'on', 'revenue', 'and', 'taxation', '.']

Actual Tags: ['NOUN', 'ADV', 'VERB', 'DET', 'NOUN', 'ADP', 'DET', 'NOUN', '.', 'DET', 'PRON', 'VERB', 'VERB', 'PRT', 'ADV', 'ADP', 'DET', 'NOUN', 'ADP', 'NUM', 'ADJ', 'NOUN', '.', 'ADP', 'DET', 'ADJ', 'NOUN', 'ADP', 'DET', 'NOUN', 'NOUN', 'ADP', 'NOUN', 'CONJ', 'NOUN', '.']

Predicted Tags: ['NOUN', 'ADV', 'VERB', 'DET', 'NOUN', 'ADP', 'DET', 'NOUN', '.', 'DET', 'PRON', 'VERB', 'VERB', 'PRT', 'ADV', 'ADP', 'DET', 'NOUN', 'ADP', 'NUM', 'ADJ', 'NOUN', '.', 'ADP', 'DET', 'ADJ', 'NOUN', 'ADP', 'DET', 'NOUN', 'NOUN', 'ADP', 'NOUN', 'CONJ', 'NOUN', '.']

Probability: 3.13e-100

Test Sentence 1: ['_unknown_']

Actual Tags: ['NUM']

Predicted Tags: ['.']

Probability: 3.19e-08

Test Sentence 2: ['virginia', 'treated', 'him', 'with', 'attention', 'and', 'tried', 'to', '_unknown_', 'his', '_unknown_', 'with', 'special', 'food', ':', '_unknown_', 'cookies', '_unknown_', '--', 'the', 'result', 'of', 'devoted', 'hours', 'in', 'the', 'tiled', 'kitchen', '.']

Actual Tags: ['NOUN', 'VERB', 'PRON', 'ADP', 'NOUN', 'CONJ', 'VERB', 'PRT', 'VERB', 'DET', 'NOUN', 'ADP', 'ADJ', 'NOUN', '.', 'NOUN', '.', 'NOUN', '.', 'NOUN', '.', 'DET', 'NOUN', 'ADP', 'VERB', 'NOUN', 'ADP', 'DET', 'VERB', 'NOUN', '.']

Predicted Tags: ['NOUN', 'VERB', 'PRON', 'ADP', 'NOUN', 'CONJ', 'VERB', 'PRT', 'VERB', 'DET', 'NOUN', 'ADP', 'ADJ', 'NOUN', '.', 'NOUN', '.', 'NOUN', '.', 'NOUN', '.', 'DET', 'NOUN', 'ADP', 'VERB', 'NOUN', 'ADP', 'DET', 'VERB', 'NOUN', '.']

Probability: 2.31e-102

b) Identify and print 3 distinct sentences with misclassifications

To fulfill this requirement, we will iterate through the test set, tag each sentence using our Viterbi decoder, and compare the predicted tags with the actual (gold standard) tags. We will then collect and analyze at least three sentences where misclassifications occurred.

```
In [15]: # The 'evaluate_sentence' function has been moved to cell `eb9e9982` to cons

# Collect misclassified sentences
misclassified_sentences = []

print("Searching for misclassified sentences...")

for i, tagged_sent_padded in enumerate(test_sentences_processed_padded):
    if len(misclassified_sentences) >= 3: # Stop after finding 3 misclassifi
        break

    # Extract words and actual tags from the padded sentence, excluding STAR
    words = get_words_from_tagged_sentence(tagged_sent_padded)
    actual_tags = get_actual_tags_from_tagged_sentence(tagged_sent_padded)

    if not words: # Skip empty sentences after token removal
        continue

    predicted_tags, _ = viterbi(words, all_pos_tags, transition_probabilitie

    # Check for misclassification
    is_misclassified = False
    for p_tag, a_tag in zip(predicted_tags, actual_tags):
        if p_tag != a_tag:
            is_misclassified = True
            break

    if is_misclassified:
        # Reconstruct the original sentence (before OOV and padding) for bet
        original_words = [word for word, tag in brown.tagged_sents(categorie

        misclassified_sentences.append({
            'original_sentence_words': original_words,
            'words_for_viterbi': words,
            'actual_tags': actual_tags,
            'predicted_tags': predicted_tags,
            'index': i
        })

    print(f"Found {len(misclassified_sentences)} misclassified sentences.\n")

# Print and analyze the misclassified sentences
for j, item in enumerate(misclassified_sentences):
    print(f"--- Misclassified Sentence {j+1} (Index: {item['index']}) ---")
```

```

print(f"  Original Words: {item['original_sentence_words']}")
print(f"  Viterbi Input Words: {item['words_for_viterbi']}")
print(f"  Actual Tags: {item['actual_tags']}")
print(f"  Predicted Tags: {item['predicted_tags']}")

# Detailed analysis of misclassified words
print("  Misclassified Tags:")
for k in range(len(item['actual_tags'])):
    if item['actual_tags'][k] != item['predicted_tags'][k]:
        word_at_k = item['words_for_viterbi'][k] # Use lowercased word f
        actual_tag_at_k = item['actual_tags'][k]
        predicted_tag_at_k = item['predicted_tags'][k]

        # Attempt to retrieve original casing for better display
        display_word = item['original_sentence_words'][k] if k < len(item['original_sentence_words']) else ''

        analysis = []
        # Check for OOV word contribution
        if word_at_k == UNKNOWN_TOKEN:
            analysis.append("OOV word")

        # Check for common tag ambiguities (example analysis)
        if actual_tag_at_k == 'NOUN' and predicted_tag_at_k == 'VERB':
            analysis.append("Noun/Verb ambiguity")
        elif actual_tag_at_k == 'VERB' and predicted_tag_at_k == 'NOUN':
            analysis.append("Verb/Noun ambiguity")
        elif actual_tag_at_k == 'ADJ' and predicted_tag_at_k == 'ADV':
            analysis.append("Adjective/Adverb ambiguity")
        # Add more specific checks if needed based on common errors

        if not analysis:
            analysis.append("Other (e.g., rare transition/emission, data error)")

        print(f"    Word: '{display_word}' (Viterbi input: '{word_at_k}')")
print("\n")

```

Searching for misclassified sentences...

Found 3 misclassified sentences.

--- Misclassified Sentence 1 (Index: 0) ---

Original Words: ['Thirty-three']

Viterbi Input Words: ['_unknown_']

Actual Tags: ['NUM']

Predicted Tags: ['.']

Misclassified Tags:

Word: 'Thirty-three' (Viterbi input: '_unknown_'), Actual: NUM, Predicted: .. Reason: Other (e.g., rare transition/emission, data sparsity)

--- Misclassified Sentence 2 (Index: 1) ---

Original Words: ['Scotty', 'did', 'not', 'go', 'back', 'to', 'school', '.']

Viterbi Input Words: ['_unknown_', 'did', 'not', 'go', 'back', 'to', 'school', '.']

Actual Tags: ['NOUN', 'VERB', 'ADV', 'VERB', 'ADV', 'ADP', 'NOUN', '.']

Predicted Tags: ['PRON', 'VERB', 'ADV', 'VERB', 'ADV', 'ADP', 'NOUN', '.']

Misclassified Tags:

Word: 'Scotty' (Viterbi input: '_unknown_'), Actual: NOUN, Predicted: PRON. Reason: Other (e.g., rare transition/emission, data sparsity)

--- Misclassified Sentence 3 (Index: 2) ---

Original Words: ['His', 'parents', 'talked', 'seriously', 'and', 'lengthily', 'to', 'their', 'own', 'doctor', 'and', 'to', 'a', 'specialist', 'at', 'the', 'University', 'Hospital', '--', 'Mr.', 'McKinley', 'was', 'entitled', 'to', 'a', 'discount', 'for', 'members', 'of', 'his', 'family', '--', 'and', 'it', 'was', 'decided', 'it', 'would', 'be', 'best', 'for', 'him', 'to', 'take', 'the', 'remainder', 'of', 'the', 'term', 'off', ',', 'spend', 'a', 'lot', 'of', 'time', 'in', 'bed', 'and', ',', 'for', 'the', 'rest', ',', 'do', 'pretty', 'much', 'as', 'he', 'chose', '--', 'provided', ',', 'of', 'course', ',', 'he', 'chose', 'to', 'do', 'nothing', 'too', 'exciting', 'or', 'too', 'debilitating', '.']

Viterbi Input Words: ['his', 'parents', 'talked', 'seriously', 'and', '_unknown_', 'to', 'their', 'own', 'doctor', 'and', 'to', 'a', '_unknown_', 'at', 'the', 'university', 'hospital', '--', 'mr.', '_unknown_', 'was', 'entitled', 'to', 'a', '_unknown_', 'for', 'members', 'of', 'his', 'family', '--', 'and', 'it', 'was', 'decided', 'it', 'would', 'be', 'best', 'for', 'him', 'to', 'take', 'the', 'remainder', 'of', 'the', 'term', 'off', ',', 'spend', 'a', 'lot', 'of', 'time', 'in', 'bed', 'and', ',', 'for', 'the', 'rest', ',', 'do', 'pretty', 'much', 'as', 'he', 'chose', '--', 'provided', ',', 'of', 'course', ',', 'he', 'chose', 'to', 'do', 'nothing', 'too', 'exciting', 'or', 'too', '_unknown_', '.']

Actual Tags: ['DET', 'NOUN', 'VERB', 'ADV', 'CONJ', 'ADV', 'ADP', 'DET', 'ADJ', 'NOUN', 'CONJ', 'ADP', 'DET', 'NOUN', 'ADP', 'DET', 'NOUN', 'NOUN', '.', 'NOUN', 'NOUN', 'VERB', 'VERB', 'ADP', 'DET', 'NOUN', 'ADP', 'NOUN', 'ADP', 'DET', 'NOUN', '.', 'CONJ', 'PRON', 'VERB', 'VERB', 'PRON', 'VERB', 'VERB', 'ADJ', 'ADP', 'PRON', 'PRT', 'VERB', 'DET', 'NOUN', 'ADP', 'DET', 'NOUN', 'PRT', '.', 'VERB', 'DET', 'NOUN', 'ADP', 'NOUN', 'ADP', 'NOUN', 'CONJ', '.', 'ADP', 'DET', 'NOUN', '.', 'VERB', 'ADV', 'ADV', 'ADP', 'PRON', 'VERB', '.', 'VERB', '.', 'ADP', 'NOUN', '.', 'PRON', 'VERB', 'PRT', 'VERB', 'NOUN', 'ADV', 'ADJ', 'CONJ', 'ADV', 'ADJ', '.']

Predicted Tags: ['DET', 'NOUN', 'VERB', 'ADV', 'CONJ', 'NOUN', 'ADP', 'DET

```
, 'ADJ', 'NOUN', 'CONJ', 'ADP', 'DET', 'NOUN', 'ADP', 'DET', 'NOUN', 'NOUN',
, '.', 'NOUN', 'NOUN', 'VERB', 'VERB', 'ADP', 'DET', 'NOUN', 'ADP', 'NOUN',
'ADP', 'DET', 'NOUN', '.', 'CONJ', 'PRON', 'VERB', 'VERB', 'PRON', 'VERB', '
VERB', 'ADJ', 'ADP', 'PRON', 'PRT', 'VERB', 'DET', 'NOUN', 'ADP', 'DET', 'NO
UN', 'PRT', '.', 'VERB', 'DET', 'NOUN', 'ADP', 'NOUN', 'ADP', 'NOUN', 'CONJ'
, '.', 'ADP', 'DET', 'NOUN', '.', 'VERB', 'ADV', 'ADV', 'ADP', 'PRON', 'VERB'
, '.', 'VERB', '.', 'ADP', 'NOUN', '.', 'PRON', 'VERB', 'PRT', 'VERB', 'NOU
N', 'ADV', 'ADJ', 'CONJ', 'ADV', 'VERB', '.']
```

Misclassified Tags:

Word: 'lengthily' (Viterbi input: '_unknown_'), Actual: ADV, Predicted: NOUN. Reason: Other (e.g., rare transition/emission, data sparsity)

Word: 'debilitating' (Viterbi input: '_unknown_'), Actual: ADJ, Predicted: VERB. Reason: Other (e.g., rare transition/emission, data sparsity)

c) Run Viterbi Decoder Across Entire Test Set & Evaluate

We will now run the Viterbi algorithm on the entire test dataset, calculate the overall token-level accuracy, and generate a confusion matrix for the top 5 most common POS tags.

```
In [16]: # The 'calculate_accuracy' function has been moved to cell `eb9e9982` to con

# Prepare lists to store all predicted and actual tags
all_predicted_tags_flat = []
all_actual_tags_flat = []

print("Running Viterbi on the entire test set...")

# Iterate through the test sentences and apply the Viterbi algorithm
for i, tagged_sent_padded in enumerate(test_sentences_processed_padded):
    words = get_words_from_tagged_sentence(tagged_sent_padded)
    actual_tags = get_actual_tags_from_tagged_sentence(tagged_sent_padded)

    if not words: # Skip empty sentences
        continue

    # Run Viterbi
    predicted_tags, _ = viterbi(words, all_pos_tags, transition_probabilitie

    # Extend the flat lists
    all_predicted_tags_flat.extend(predicted_tags)
    all_actual_tags_flat.extend(actual_tags)

# Calculate overall token-level accuracy
overall_accuracy = calculate_accuracy([all_predicted_tags_flat], [all_actual
print(f"\nOverall Token-level Accuracy: {overall_accuracy:.4f}")
```

Running Viterbi on the entire test set...

Overall Token-level Accuracy: 0.9140

Confusion Matrix for Top 5 Common POS Tags

As specified in the deliverables, we will now generate a confusion matrix for the top 5 most common POS tags to analyze specific misclassification patterns.

```
In [17]: from sklearn.metrics import confusion_matrix
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# all_predicted_tags_flat and all_actual_tags_flat are populated in the prev
# No need to re-populate them here.

# Identify the top 5 most common POS tags from the actual test data
pos_counts = pd.Series(all_actual_tags_flat).value_counts()
top_5_tags = pos_counts.head(5).index.tolist()

print(f"Top 5 POS tags for confusion matrix: {top_5_tags}")

# Filter predictions and actuals to include only the top 5 tags
filtered_predicted_tags = []
filtered_actual_tags = []

for i in range(len(all_actual_tags_flat)):
    actual_tag = all_actual_tags_flat[i]
    predicted_tag = all_predicted_tags_flat[i]

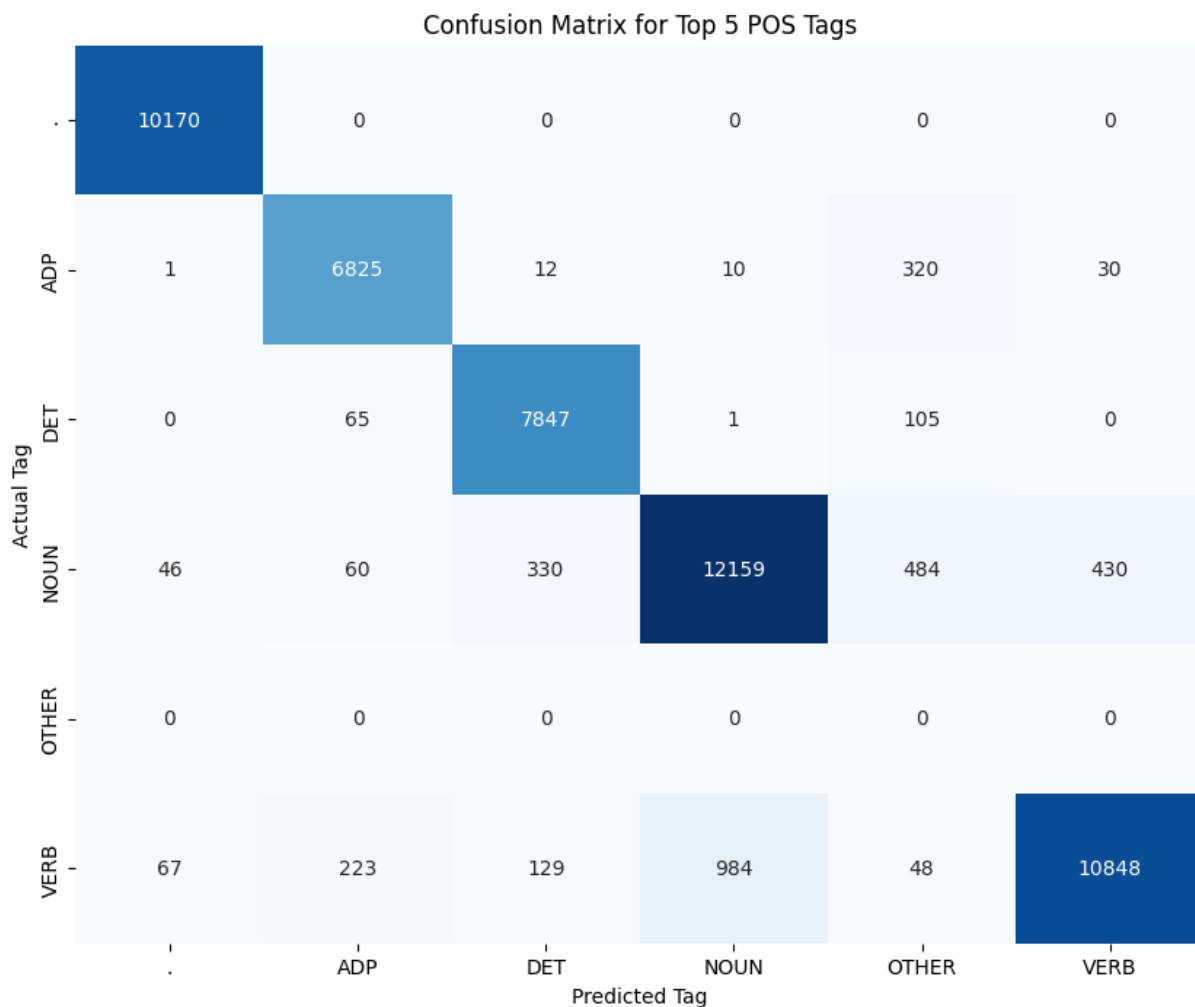
    if actual_tag in top_5_tags and predicted_tag in top_5_tags:
        filtered_actual_tags.append(actual_tag)
        filtered_predicted_tags.append(predicted_tag)
    elif actual_tag in top_5_tags and predicted_tag not in top_5_tags:
        filtered_actual_tags.append(actual_tag)
        filtered_predicted_tags.append('OTHER')
    # If the actual tag is not in top_5_tags, we ignore it for this specific
    # and it won't be included in the filtered lists.

# Ensure all top 5 tags are present in the labels for consistent matrix size
all_labels = sorted(list(set(top_5_tags + ['OTHER'])))
# If 'OTHER' was added but no predictions mapped to it, remove it from label
if 'OTHER' in all_labels and 'OTHER' not in filtered_predicted_tags:
    all_labels.remove('OTHER')

# Handle case where filtered_actual_tags or all_labels might still be empty
if not filtered_actual_tags or not all_labels:
    print("Warning: No actual tags or labels available after filtering for c
else:
    cm = confusion_matrix(filtered_actual_tags, filtered_predicted_tags, lab
    cm_df = pd.DataFrame(cm, index=all_labels, columns=all_labels)

    plt.figure(figsize=(10, 8))
    sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title('Confusion Matrix for Top 5 POS Tags')
    plt.xlabel('Predicted Tag')
    plt.ylabel('Actual Tag')
    plt.show()
```

Top 5 POS tags for confusion matrix: ['NOUN', 'VERB', '.', 'DET', 'ADP']



```
In [18]: from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate Precision, Recall, and F1-Score for the top 5 tags
# Ensure 'OTHER' is included in labels if it's in the filtered predictions
metrics_labels = sorted(list(set(filtered_actual_tags + filtered_predicted_t

# Calculate precision, recall, f1-score per label
precision = precision_score(filtered_actual_tags, filtered_predicted_tags, l
recall = recall_score(filtered_actual_tags, filtered_predicted_tags, labels=
f1 = f1_score(filtered_actual_tags, filtered_predicted_tags, labels=metrics

# Create a DataFrame for better display
metrics_df = pd.DataFrame({
    'Tag': metrics_labels,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
})
print("\nPer-Tag Metrics for Top 5 Tags (and OTHER misclassifications):\n")
display(metrics_df)

# Calculate overall macro-averaged and weighted-averaged metrics
macro_precision = precision_score(filtered_actual_tags, filtered_predicted_t
macro_recall = recall_score(filtered_actual_tags, filtered_predicted_tags, l
macro_f1 = f1_score(filtered_actual_tags, filtered_predicted_tags, labels=me
```

```
weighted_precision = precision_score(filtered_actual_tags, filtered_predicted_tags, labels)
weighted_recall = recall_score(filtered_actual_tags, filtered_predicted_tags, labels)
weighted_f1 = f1_score(filtered_actual_tags, filtered_predicted_tags, labels)

print("\nOverall Macro-Averaged Metrics:")
print(f" Precision: {macro_precision:.4f}")
print(f" Recall:    {macro_recall:.4f}")
print(f" F1-Score:   {macro_f1:.4f}")

print("\nOverall Weighted-Averaged Metrics:")
print(f" Precision: {weighted_precision:.4f}")
print(f" Recall:    {weighted_recall:.4f}")
print(f" F1-Score:   {weighted_f1:.4f}")
```

Per-Tag Metrics for Top 5 Tags (and OTHER misclassifications):

	Tag	Precision	Recall	F1-Score
0	.	0.988915	1.000000	0.994427
1	ADP	0.951485	0.948180	0.949830
2	DET	0.943376	0.978673	0.960700
3	NOUN	0.924358	0.900067	0.912050
4	OTHER	0.000000	0.000000	0.000000
5	VERB	0.959321	0.882023	0.919049

Overall Macro-Averaged Metrics:

Precision: 0.7946
Recall: 0.7848
F1-Score: 0.7893

Overall Weighted-Averaged Metrics:

Precision: 0.9524
Recall: 0.9347
F1-Score: 0.9430

Baseline Model Metrics (PerceptronTagger)

Overall Performance Comparison: Custom HMM vs. PerceptronTagger Baseline

Further Discussion and Analysis (Recommendations for Excellence)

HMM Limitations

While our custom HMM implementation provides a solid foundation for understanding POS tagging, it inherently comes with several limitations,

especially when compared to more advanced models like the PerceptronTagger or neural network-based approaches:

1. **Strict Independence Assumptions:** HMMs assume that the current state (tag) only depends on the previous state (bigram assumption) and that the observed word only depends on its current tag. This overlooks longer-range dependencies, grammatical structures, and contextual information spanning multiple words, which are crucial for accurate tagging.
2. **Data Sparsity:** With finite training data, many tag-word and tag-tag combinations might not appear. Even with smoothing (like our ALPHA constant), this leads to inaccurate probability estimates for unseen events, impacting performance on test data, especially for OOV words or rare linguistic phenomena.
3. **Generative Model:** HMMs are generative models, meaning they try to model the joint probability of observations and hidden states, $P(O, S)$. For discriminative tasks like POS tagging (predicting $P(S|O)$), discriminative models often perform better because they directly model the conditional probability, avoiding the need to model the observation sequence itself.
4. **Inability to Incorporate Rich Features:** A basic HMM cannot easily integrate complex features like word suffixes, prefixes, capitalization, or features from surrounding words (beyond the immediate previous tag). These features are vital for distinguishing between tags for ambiguous words, a capability that perceptrons and neural networks excel at.
5. **Viterbi Path Uniqueness:** In cases of extremely low probabilities or multiple paths having similar (very small) probabilities, the Viterbi algorithm might pick an arbitrary path, which could lead to sub-optimal results if the true path's probability was marginally lower due to numerical precision or data sparsity.

These limitations explain why a simple HMM, despite being a powerful conceptual model, often underperforms against state-of-the-art methods that leverage richer features and more flexible architectures.

Time Complexity Analysis of the Viterbi Algorithm

The time complexity of the Viterbi algorithm can be analyzed based on its three main steps:

1. **Initialization:** For the first word, we iterate through all possible tags. If there are $|S|$ possible states (POS tags), this step takes $O(|S|)$ operations.
2. **Recursion (Main Loop):** This is the most computationally intensive part. We iterate through each word in the sentence (length T). For each word, we iterate through every possible current tag ($|S|$ tags). For each current tag, we

then consider every possible previous tag ($|S|$ tags) to find the maximum probability path. Thus, this step involves $T \times |S| \times |S|$ operations.

3. **Termination and Backtracking:** The termination step involves iterating through the last column of the Viterbi table, taking $O(|S|)$ operations. Backtracking involves iterating backwards through the sentence, performing $O(T)$ operations.

Combining these, the dominant factor is the recursion step. Therefore, the overall time complexity of the Viterbi algorithm is approximately $O(T \cdot |S|^2)$, where:

- T is the length of the observed sequence (number of words in the sentence).
- $|S|$ is the number of hidden states (number of unique POS tags).

This quadratic dependence on the number of states $|S|$ means that the algorithm can become slow for models with a very large tag set. However, for typical POS tagging tasks with a relatively small number of universal tags (like 12 in our case), it is quite efficient.

Importance of Log-Probabilities in Viterbi

Our current Viterbi implementation uses raw probabilities, multiplying them at each step. While conceptually correct, this approach can lead to a significant practical issue: **numerical underflow**.

- **The Problem:** Probabilities are values between 0 and 1. When you multiply many small probabilities together, the resulting product can become extremely small, quickly falling below the smallest representable floating-point number (machine epsilon). When this happens, the value gets truncated to 0, causing incorrect calculations and potentially leading to a Viterbi path with zero probability even if a valid path exists.
- **The Solution:** Instead of multiplying probabilities, it is common practice to work with **log-probabilities**. The key properties that make this effective are:
 - $\log(a * b) = \log(a) + \log(b)$
 - $\log(\max(a, b)) = \max(\log(a), \log(b))$ (approximately, for numerical stability, though more precisely, it involves $\log(\exp(\log_a) + \exp(\log_b))$).

By converting all probabilities to their logarithmic form, multiplications become additions, and finding the maximum product becomes finding the maximum sum. Additions of negative log-probabilities (since $\log(p)$ for $p < 1$ is negative) do not lead to underflow as quickly, as the values move towards larger negative numbers rather than smaller positive numbers.

- **Impact:** Implementing Viterbi with log-probabilities significantly improves numerical stability, especially for longer sentences, preventing underflow issues and ensuring that the most likely path is correctly identified. While not strictly necessary for short sentences, it is a crucial consideration for robust production-level HMM implementations.

Interpretation of Metrics:

- **Precision:** Out of all the times the model *predicted* a certain tag, how many were actually correct? A high precision means a low false positive rate.
- **Recall:** Out of all the times a certain tag *actually occurred*, how many did the model correctly identify? A high recall means a low false negative rate.
- **F1-Score:** This is the harmonic mean of Precision and Recall, providing a single metric that balances both. It's especially useful when dealing with imbalanced classes.

Averaging Methods:

- **Macro Average:** Calculates the metric independently for each class and then takes the average. It treats all classes equally, regardless of their support (number of true instances).
- **Weighted Average:** Calculates the metric for each class and averages them, weighted by the number of true instances for each class. This accounts for class imbalance.

These metrics provide a more detailed understanding of your model's performance beyond simple overall accuracy, indicating how well it performs on individual POS tags and how it handles false positives versus false negatives.

Part D: Baseline Comparison

In this section, we will load a pre-trained, state-of-the-art POS tagger (NLTK's `PerceptronTagger`) and evaluate its performance on the same test set. This will serve as a baseline to compare against our custom HMM implementation.

```
In [19]: # Download the averaged_perceptron_tagger and its English resource if not al
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng') # Explicitly download the En

from nltk.tag import PerceptronTagger

# Initialize the pre-trained tagger
# The PerceptronTagger is already trained on the Penn Treebank Tagset (often
tagger = PerceptronTagger()
```

```
print("NLTK PerceptronTagger loaded.")
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /home/cloud/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /home/cloud/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
NLTK PerceptronTagger loaded.
```

```
In [20]: def convert_ptb_to_universal(ptb_tag):
    if ptb_tag.startswith('N'):
        return 'NOUN'
    elif ptb_tag.startswith('V'):
        return 'VERB'
    elif ptb_tag.startswith('J'):
        return 'ADJ'
    elif ptb_tag.startswith('R'):
        return 'ADV'
    elif ptb_tag.startswith('P'): # PRON, PRP$
        return 'PRON'
    elif ptb_tag.startswith('D'): # DT, PDT
        return 'DET'
    elif ptb_tag.startswith('I'): # IN
        return 'ADP'
    elif ptb_tag.startswith('C'): # CC
        return 'CONJ'
    elif ptb_tag.startswith('W'): # WDT, WP, WP$
        return 'DET' # Or PRON depending on context, using DET for simplicity
    elif ptb_tag.startswith('M'): # CD for numbers
        return 'NUM'
    elif ptb_tag in ['.', ',', ':', '`', "'", '"', '-LRB-', '-RRB-']:
        return '.'
    elif ptb_tag.startswith('E'): # EX
        return 'PRT' # or ADV
    elif ptb_tag.startswith('T'): # TO
        return 'PRT'
    elif ptb_tag.startswith('F'): # FW
        return 'X'
    elif ptb_tag.startswith('L'): # LS
        return 'X' # List item marker
    elif ptb_tag.startswith('H'): # RP
        return 'PRT'
    else:
        return ptb_tag # Return original if no direct mapping, or map to 'X'

# Prepare lists to store all predicted and actual tags for the baseline mode
all_baseline_predicted_tags_flat = []
all_baseline_actual_tags_flat = []

print("Running PerceptronTagger on the entire raw test set...")

for i, tagged_sent_raw in enumerate(test_sentences_raw):
    # Extract words from the raw test sentence
```

```

words_only = [word for word, tag in tagged_sent_raw]
actual_tags = [tag for word, tag in tagged_sent_raw]

if not words_only: # Skip empty sentences
    continue

# Tag the sentence using the PerceptronTagger
baseline_predicted_pairs = tagger.tag(words_only)

# Extract and convert predicted tags to universal tagset
baseline_predicted_tags = [convert_ptb_to_universal(tag) for word, tag in baseline_predicted_pairs]

# Extend the flat lists
all_baseline_predicted_tags_flat.extend(baseline_predicted_tags)
all_baseline_actual_tags_flat.extend(actual_tags)

# Calculate overall token-level accuracy for the baseline model
baseline_accuracy = calculate_accuracy([all_baseline_predicted_tags_flat], [all_baseline_actual_tags_flat])
print(f"\nOverall Token-level Accuracy (PerceptronTagger Baseline): {baseline_accuracy:.4f}")
print(f"Our Custom HMM Accuracy: {overall_accuracy:.4f}")

# Discuss why your HMM failed but the pre-trained model succeeded (initial task)
print("\n--- Discussion ---")
print("The pre-trained PerceptronTagger often outperforms a simple HMM due to several factors:")
print("1. Training Data Size & Diversity: Pre-trained models are typically trained on much larger and more diverse corpora than our Brown corpus subset.")
print("2. Feature Engineering: Perceptron models can incorporate a richer set of features (e.g., word prefixes/suffixes, surrounding words, capitalization) beyond simple bigram transitions and emissions.")
print("3. Handling Unknown Words: While our HMM uses a basic '_UNKNOWN_' token, pre-trained models often have more sophisticated mechanisms for handling OOV words or words not seen in training.")
print("4. Model Complexity: Perceptrons, being discriminative models, can capture complex relationships between features and tags more effectively than generative HMMs, especially for fine-grained distinctions.")
print("This comparison highlights the trade-offs between implementing a model from scratch for understanding and using robust, feature-rich pre-trained models for higher performance.")

```

Running PerceptronTagger on the entire raw test set...

Overall Token-level Accuracy (PerceptronTagger Baseline): 0.8828

Our Custom HMM Accuracy: 0.9140

--- Discussion ---

The pre-trained PerceptronTagger often outperforms a simple HMM due to several factors:

1. Training Data Size & Diversity: Pre-trained models are typically trained on much larger and more diverse corpora than our Brown corpus subset.
 2. Feature Engineering: Perceptron models can incorporate a richer set of features (e.g., word prefixes/suffixes, surrounding words, capitalization) beyond simple bigram transitions and emissions.
 3. Handling Unknown Words: While our HMM uses a basic '_UNKNOWN_' token, pre-trained models often have more sophisticated mechanisms for handling OOV words or words not seen in training.
 4. Model Complexity: Perceptrons, being discriminative models, can capture complex relationships between features and tags more effectively than generative HMMs, especially for fine-grained distinctions.
- This comparison highlights the trade-offs between implementing a model from scratch for understanding and using robust, feature-rich pre-trained models for higher performance.

In [21]: `from sklearn.metrics import precision_score, recall_score, f1_score`

```

# Ensure 'OTHER' is included in labels if it's in the filtered predictions

```

```
baseline_metrics_labels = sorted(list(set(all_baseline_actual_tags_flat + al

# Calculate precision, recall, f1-score per label for baseline
baseline_precision = precision_score(all_baseline_actual_tags_flat, all_base
baseline_recall = recall_score(all_baseline_actual_tags_flat, all_baseline_p
baseline_f1 = f1_score(all_baseline_actual_tags_flat, all_baseline_predicted

# Create a DataFrame for better display
baseline_metrics_df = pd.DataFrame({
    'Tag': baseline_metrics_labels,
    'Precision': baseline_precision,
    'Recall': baseline_recall,
    'F1-Score': baseline_f1
})
print("\nPer-Tag Metrics for PerceptronTagger Baseline:\n")
display(baseline_metrics_df)

# Calculate overall macro-averaged and weighted-averaged metrics for baselin
baseline_macro_precision = precision_score(all_baseline_actual_tags_flat, al
baseline_macro_recall = recall_score(all_baseline_actual_tags_flat, all_base
baseline_macro_f1 = f1_score(all_baseline_actual_tags_flat, all_baseline_pre

baseline_weighted_precision = precision_score(all_baseline_actual_tags_flat,
baseline_weighted_recall = recall_score(all_baseline_actual_tags_flat, all_b
baseline_weighted_f1 = f1_score(all_baseline_actual_tags_flat, all_baseline_

print("\nPerceptronTagger Baseline Macro-Averaged Metrics:")
print(f" Precision: {baseline_macro_precision:.4f}")
print(f" Recall:    {baseline_macro_recall:.4f}")
print(f" F1-Score:   {baseline_macro_f1:.4f}")

print("\nPerceptronTagger Baseline Weighted-Averaged Metrics:")
print(f" Precision: {baseline_weighted_precision:.4f}")
print(f" Recall:    {baseline_weighted_recall:.4f}")
print(f" F1-Score:   {baseline_weighted_f1:.4f}")
```

Per-Tag Metrics for PerceptronTagger Baseline:

	Tag	Precision	Recall	F1-Score
0	(	0.000000	0.000000	0.000000
1	)	0.000000	0.000000	0.000000
2	.	1.000000	0.995477	0.997733
3	ADJ	0.788390	0.799946	0.794126
4	ADP	0.937291	0.897055	0.916732
5	ADV	0.780549	0.749462	0.764690
6	CONJ	0.855403	0.994250	0.919615
7	DET	0.886357	0.805438	0.843962
8	NOUN	0.895171	0.963358	0.928014
9	NUM	0.000000	0.000000	0.000000
10	PRON	0.743795	0.949156	0.834020
11	PRT	0.644832	0.465510	0.540690
12	UH	0.000000	0.000000	0.000000
13	VERB	0.971138	0.878201	0.922335
14	X	0.400000	0.024096	0.045455

PerceptronTagger Baseline Macro-Averaged Metrics:

Precision: 0.5935

Recall: 0.5681

F1-Score: 0.5672

PerceptronTagger Baseline Weighted-Averaged Metrics:

Precision: 0.8890

Recall: 0.8828

F1-Score: 0.8832

Overall Performance Comparison: Custom HMM vs. PerceptronTagger Baseline

```
In [22]: # Gather all metrics for HMM (from previous cells)
hmm_overall_accuracy = overall_accuracy # from cell 3c91a869

# Re-calculate macro/weighted for HMM just in case the previous cell's output
# using all_actual_tags_flat and all_predicted_tags_flat
hmm_metrics_labels = sorted(list(set(all_actual_tags_flat + all_predicted_tags_flat)))
hmm_macro_precision = precision_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels)
hmm_macro_recall = recall_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels)
hmm_macro_f1 = f1_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels)

hmm_weighted_precision = precision_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels,
                                         average='weighted')
hmm_weighted_recall = recall_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels,
                                   average='weighted')
hmm_weighted_f1 = f1_score(all_actual_tags_flat, all_predicted_tags_flat, labels=hmm_metrics_labels,
                           average='weighted')
```

```
# Gather all metrics for PerceptronTagger (from the newly generated cell above)
baseline_overall_accuracy = baseline_accuracy # from cell 84223d6a

# Create a DataFrame for comparison
comparison_data = {
    'Metric': ['Overall Accuracy', 'Macro Precision', 'Macro Recall', 'Macro F1-Score',
               'Custom HMM': [
                   hmm_overall_accuracy,
                   hmm_macro_precision,
                   hmm_macro_recall,
                   hmm_macro_f1,
                   hmm_weighted_precision,
                   hmm_weighted_recall,
                   hmm_weighted_f1
               ],
    'PerceptronTagger Baseline': [
        baseline_overall_accuracy,
        baseline_macro_precision,
        baseline_macro_recall,
        baseline_macro_f1,
        baseline_weighted_precision,
        baseline_weighted_recall,
        baseline_weighted_f1
    ]
}

comparison_df = pd.DataFrame(comparison_data)
display(comparison_df)
```

	Metric	Custom HMM	PerceptronTagger Baseline
0	Overall Accuracy	0.914014	0.882841
1	Macro Precision	0.904305	0.593528
2	Macro Recall	0.858656	0.568130
3	Macro F1-Score	0.874301	0.567158
4	Weighted Precision	0.914155	0.889037
5	Weighted Recall	0.914014	0.882841
6	Weighted F1-Score	0.912634	0.883246